MGU BCA - 3RD SEM

Python Programming

Complete Important Questions

By Aben Sojan

Exam - 16 Nov 2026

MODULE 1

Introduction, Strings, Control Flow

1

EXAM PATTERN (from real MGU question papers)

Exam: MG3SECBCA200 Python Programming - 50 marks, 1 hour 30 minutes (MGU Honours model paper, mgu.ac.in, Oct 2025).

Part A: 10 x 2 = 20 marks, answer ALL - short questions from every module.

Part B: answer any 3 of 5 x 5 = 15 marks - programs and detailed concept answers.

Part C: answer any 1 of 2 x 15 = 15 marks - one long essay or program.

Module 1 in the model paper: indentation, = vs ==, slicing (Part A); leap year program (Part B); loop control statements (Part C).

Every module, every important, repeated and sure-mark question for Python Programming, each with a full-mark model answer.

1 Explain the key features of Python. (MG2DSCCSC100, Nov 2025)

REPEATED

Python is simple and easy to learn: its syntax reads almost like plain English, so beginners write working programs quickly.

Python is interpreted: code runs line by line without a separate compilation step, which makes testing and debugging fast.

Python is free and open source: anyone can download, use and modify it at no cost.

Python is portable: the same program runs on Windows, Linux and Mac with little or no change.

Python has a huge standard library: ready-made modules exist for files, maths, dates, web work and more.

Python is dynamically typed: you never declare variable types, Python decides the type at run time.

Example:

```
x = 10
print(type(x))   # dynamically typed: no declaration needed
```

A large community and libraries like Numpy and Django mean almost every problem already has a Python solution.

These features are why Python is the first choice for beginners and for fields like data science and AI.

2 Explain the application areas of Python.

IMPORTANT

Python is a general-purpose language, so it is used in many different fields.

Web development: sites and web apps are built with frameworks like Django and Flask, which handle pages, users and databases quickly.

Data science and AI: Numpy, Pandas and scikit-learn are used for data analysis and machine learning models.

Automation and scripting: short scripts do boring repetitive tasks like renaming files, sending emails or scraping web pages automatically.

Education and research: simple English-like syntax makes it the first language taught in many schools and widely used by scientists.

Game development: libraries like Pygame help build simple 2D games.

Desktop applications: tools like Tkinter and PyQt create GUI apps that run on the computer.

Cybersecurity: Python is used to write security tools, scanners and penetration-testing scripts.

So from websites to robots, Python fits almost everywhere, which is why companies everywhere hire Python developers.

3 Why is Python called an interpreted language?

IMPORTANT

An interpreted language is one whose code is run line by line by a program called an interpreter, without a separate compilation step.

A compiled language like C must first be fully translated into machine code by a compiler before it can run.

In Python, the source code is first converted into an intermediate form called bytecode.

Then the Python Virtual Machine (PVM) reads the bytecode and executes it line by line.

You can even type commands one by one in the interactive shell (the >>> prompt) and see each result at once.

This means you can run a program, see an error, fix it and run again immediately, with no waiting for compilation.

That is why testing and development in Python is fast and beginner friendly.

So Python is called an interpreted language because its code is executed by an interpreter (the PVM) instead of being compiled to machine code first.

4 What are the different ways to run Python?

IMPORTANT

Interactive mode: type python3 in the terminal and run one line at a time; best for quick experiments.

Script mode: write code in a .py file and run it with python3 file.py; the normal way to run full programs.

IDEs and editors: tools like VS Code, PyCharm or IDLE run Python with debugging and code completion.

Jupyter notebooks: code runs in cells with output shown below each cell; popular in data science.

Online interpreters: websites that run Python in the browser with nothing installed.

For exams, remember interactive mode and script mode as the two basic ways of running Python.

5 What is the extension of a Python file?

IMPORTANT

Python source files use the .py extension, for example program.py.

The .py extension tells the operating system and the interpreter that the file holds Python code.

Related extensions: .pyc for compiled bytecode files that Python creates automatically, and .pyw for windowed scripts on Windows.

In answers and file handling questions, always name Python files with the .py extension.

6 Who developed Python and when was it released?

Python was developed by Guido van Rossum, a Dutch programmer.

He started the project in the late 1980s and released the first version, Python 0.9.0, in 1991.

Python was named after the British comedy show Monty Python, not the snake.

Van Rossum led the language for many years as its "Benevolent Dictator For Life" (BDFL) until 2018.

Because Python was free and open source, thousands of developers around the world contributed to it.

The release of Python 2.0 (2000) and Python 3.0 (2008) made it a major modern language.

Today Python is one of the most popular languages in the world.

Remember three facts for full marks: creator = Guido van Rossum, year = 1991, name from = the Monty Python show.

7 What is an identifier? State the rules for naming identifiers.

IMPORTANT

An identifier is a name given to program elements like variables, functions, classes and modules.

Example: in total = 10, the name total is an identifier.

Rule 1: it can contain letters (a-z, A-Z), digits (0-9) and underscores (_).

Rule 2: it must start with a letter or an underscore, never with a digit, so _name and name1 are valid but 1name is not.

Rule 3: it cannot be a keyword like if, for, while or class, because those have fixed meanings in the language.

Rule 4: Python is case sensitive, so Name and name are two different identifiers.

Rule 5: no spaces or special symbols like @, - or $ are allowed in a name.

Good practice: use meaningful names like total_marks instead of single letters like x, so the code is easy to read.

8 What are keywords? Give any five examples.

IMPORTANT

Keywords are reserved words in Python that have a fixed meaning and purpose.

They are part of the language itself, so you cannot use them as variable or function names.

Five examples: if (decision making), else (alternative branch), for (looping), while (condition loop) and def (defining a function).

More examples: return, import, class, True, False and None.

If you try to write name = if, Python gives a syntax error because if is reserved.

There are about 35 keywords in Python 3, and the list is fixed for the language.

You can see all of them by running help("keywords") in the Python shell.

In short: keywords belong to Python, so programmers must pick other names for their own variables.

9 Explain the structure of a Python program.

IMPORTANT

A Python program is a set of statements executed one after another from top to bottom.

A statement is a single instruction, like x = 10.

Related statements are grouped into a block using the same indentation (spaces at the start of the line).

Python needs no semicolons to end lines and no curly braces for blocks, unlike C or Java.

A typical program has: comments, import statements, variable assignments, function definitions and the main logic.

Example code:

```
x = 10
if x > 5:
print("big")
```

Here the print statement is inside the if block because it is indented one level deeper.

If the indentation is wrong, Python gives an IndentationError, so neat spacing is part of the structure.

10 Demonstrate the significance of indentation in Python programming.

IMPORTANT

In most languages braces mark a block of code; in Python, indentation (spaces at the line start) does that job.

The interpreter decides which statements belong to an if, for, while or function body purely from their indentation.

Wrong or inconsistent indentation causes an IndentationError and the program will not run at all.

Standard practice is 4 spaces per level; mixing tabs and spaces in one file is an error.

Example:

```
age = 18
if age >= 18:
print("Adult")      # indented: inside the if block
print("Done")           # not indented: outside the if block
```

So indentation in Python is not decoration; it is syntax that defines the structure of the program.

11 Distinguish between / and // and between = and ==. (Model Paper, Mar 2024)

REPEATED

/ is normal division and always returns a float, even for whole results: 7 / 2 gives 3.5.

// is floor division: it divides and drops the fractional part, rounding down: 7 // 2 gives 3.

Note the rounding direction: -7 // 2 gives -4, because floor means the next lower whole number.

= is the assignment operator: it stores a value in a variable. Example: x = 5 puts 5 into x.

== is the comparison operator: it checks whether two values are equal and returns True or False.

Example: x == 5 tests the value of x; if x is 5 the result is True.

A common beginner mistake is writing if x = 5 instead of if x == 5; the first one is a syntax error because assignment is not a test.

Remember the trick: single = stores, double == compares.

12 Explain the different types of operators in Python with examples.

IMPORTANT

An operator is a symbol that performs an operation on values called operands.

Arithmetic operators do maths: + - * / // % **. Example: 7 // 2 gives 3, because floor division drops the fractional part.

Comparison operators compare values and give True or False: == != > < >= <=. Example: 5 > 3 gives True.

Logical operators combine conditions: and, or, not. Example: (5 > 3) and (2 < 4) gives True.

Assignment operators store values: = stores; += -= *= /= update and store back. Example: x = 5 then x += 2 makes x equal 7.

Identity operators: is and is not check whether two names point to the same object in memory.

Membership operators: in and not in check whether a value exists inside a sequence. Example: "a" in "cat" gives True.

Bitwise operators (&, |, ^, <<, >>) work on the binary digits of numbers.

These operator families cover almost every operation you will write in a Python program.

13 What is operator precedence? Evaluate: 2 + 3 * 4 ** 2.

IMPORTANT

Operator precedence is the rule that decides which operator is evaluated first in an expression.

Python follows this order: ** (power) first, then * / // %, then + -, then comparison operators, then not, and, or last.

Brackets can force any order you want.

Now evaluate 2 + 3 * 4 ** 2 step by step.

Step 1: power first, 4 ** 2 = 16.

Step 2: multiplication next, 3 * 16 = 48.

Step 3: addition last, 2 + 48 = 50.

So the final answer is 50.

If we wanted addition first we would write (2 + 3) * 4 ** 2, which gives 5 * 16 = 80, showing why precedence matters.

14 What are the purposes of the // and % operators in Python?

IMPORTANT

// is floor division: it divides and drops the fractional part, giving how many whole times one number fits in another.

% is the modulus operator: it gives the remainder left after division.

Example:

```
print(17 // 5)   # 3, because 5 fits 3 full times in 17
print(17 % 5)    # 2, the remainder
```

Common uses: // for splitting items into full groups, % for checking even or odd with n % 2 == 0.

Note the difference from / which gives the exact float result: 17 / 5 = 3.4.

In short: // answers how many full times, % answers what is left over.

15 Explain the characteristics of membership operators in Python.

IMPORTANT

Membership operators test whether a value exists inside a sequence like a string, list or tuple.

in returns True when the value is found, for example 'a' in 'apple' is True.

not in returns True when the value is absent, for example 'z' not in 'apple' is True.

They work on strings (substring check), lists, tuples, dictionaries (checks keys) and sets.

Example:

```
fruits = ["apple", "mango"]
print("mango" in fruits)        # True
print("grape" not in fruits)    # True
```

Membership operators are the standard way to check presence before using or removing an item.

16 Write a Python program to print the result of 5+3*2 and then modify it to get 16.

IMPORTANT

Python follows operator precedence: * is evaluated before +, like BODMAS in maths.

So 5+3*2 is computed as 5+(3*2) = 5+6 = 11.

To force the addition first and get 16, use brackets: (5+3)*2 = 8*2 = 16.

```
print(5+3*2)      # 11, because * runs before +
print((5+3)*2)    # 16, brackets change the order
```

Brackets always win over the default precedence order.

17 Explain the truth tables of and, or and not in Python with examples.

IMPORTANT

and gives True only when both sides are True: True and True is True, every other combination is False.

or gives True when at least one side is True: only False or False is False.

not flips the value: not True is False and not False is True.

Example:

```
print(True and False)   # False
print(True or False)    # True
print(not True)         # False
```

In programs they combine conditions, for example: if age >= 18 and has_id: allow entry.

Remember: and needs both true, or needs one true, not reverses the result.

18 Explain the basic data types in Python.

IMPORTANT

A data type tells Python what kind of value is stored and what operations are allowed on it.

int: whole numbers, positive or negative, e.g. 10, -5.

float: decimal numbers, e.g. 3.14, -0.5.

complex: complex numbers with real and imaginary parts, e.g. 2 + 3j.

str: text written in quotes, e.g. "hello".

bool: logical values, only True or False, used in conditions.

Python finds the type automatically; you do not declare it like in C or Java.

Use type() to check: type(10) gives int and type("hi") gives str.

Knowing the type matters because only matching types can be combined in operations.

19 What is type conversion? Explain with examples.

IMPORTANT

Type conversion means changing a value from one data type to another.

Python gives built-in functions for this: int(), float(), str() and bool().

int("10") converts the string "10" to the integer 10.

float("3.5") gives 3.5, and float(7) gives 7.0.

str(100) converts the number 100 to the string "100".

This is needed because input() always returns a string, even when the user types digits.

Example code:

```
age = int(input("Enter age: "))
```

Without int(), age + 1 would fail because you cannot add a number to a string.

So type conversion connects user input with numeric calculations.

20 **What will be the output of int('25') + float('2.5')? Explain.**

IMPORTANT

int("25") converts the string "25" into the integer 25.

float("2.5") converts the string "2.5" into the decimal number 2.5.

Now the expression is 25 + 2.5, with both operands being numbers, so Python adds them.

When an int and a float are mixed, Python promotes the result to float.

25 + 2.5 = 27.5, so the output is 27.5.

Without conversion, "25" + "2.5" would join the two strings into "252.5" instead of adding them.

So conversion functions like int() and float() turn string data into real numbers first.

This example shows conversion turning string data into numbers before maths can happen.

21 **Distinguish between a statement and an expression with examples.**

IMPORTANT

A statement is a complete instruction that performs an action, like x = 10.

An expression is a combination of values, variables and operators that produces a value, like 2 + 3 * 4.

Every expression evaluates to a single value; for example 2 + 3 gives the value 5.

A statement may or may not produce a value: x = 10 performs the action of storing, but gives no value back.

Example: in y = 2 + 3, the whole line is a statement, and 2 + 3 inside it is an expression.

print("hi") is a statement that displays output; "hello" + "hi" is an expression that builds the value "hellohi".

print(2 + 3) shows 5, because the expression 2 + 3 is evaluated first and its value is printed.

In short: expressions calculate values, statements do things.

22 Explain input() and print() with examples.

IMPORTANT

print() displays output on the screen.

It can show text, numbers and variables together: print("Hello", 2026) prints Hello 2026.

input() reads one line of text typed by the user and returns it.

Example: name = input("Enter name: ") waits for typing and stores it in name.

The text inside input() is called the prompt; it tells the user what to type.

Important: input() always returns a string, even if the user types digits.

So for numbers you must convert the result:

```
age = int(input("Enter age: "))
print("Next year you will be", age + 1)
```

Together, input() takes data in and print() sends results out, which is how a program talks to the user.

23 Why do we write int(input()) instead of only input()?

IMPORTANT

input() alone always returns a string, and strings cannot be used in maths directly.

If the user types 5, input() gives the string "5", not the number 5.

Adding strings joins them: "5" + "3" gives "53", not 8.

int(input()) converts the typed text into an integer first, so numeric operations work correctly.

Example: marks = int(input("Enter marks: ")) followed by total = marks + 10 works as real maths.

For decimal input use float(input()) instead of int(input()).

A good rule: use int(input()) for whole numbers, float(input()) for decimals, and plain input() for names and text.

So we wrap input() with int() whenever the program needs to calculate with the entered value.

24 **Explain string slicing with examples. (Model Paper, MG2DSCCSC100, Mar 2024)**

SURE MARK

Slicing picks out a part of a string using the form s[start:end].

It takes characters from index start up to, but not including, index end.

Indexing starts at 0, so s[0] is the first character of the string.

Example code:

```
s = 'Python'
print(s[1:4])
```

Indexes 1, 2, 3 hold y, t, h, so the output is yth.

Shortcuts: s[:3] takes from the start and gives Pyt; s[2:] takes to the end and gives thon.

s[:] copies the whole string; s[::-1] reverses it and gives nohtyP.

Negative indexes count from the end: s[-1] is n, and s[-4:-1] gives tho.

Slicing never changes the original string; it returns a new one, because strings are immutable.

25 **Explain any five built-in string functions with examples.**

IMPORTANT

Strings come with many built-in methods for common text jobs.

len("hello") gives 5, the number of characters in the string.

"hello".upper() gives HELLO; "HELLO".lower() gives hello; "hello".title() gives Hello.

" a b ".strip() removes spaces from both ends, giving "a b".

"a,b,c".split(",") breaks the string at each comma and gives the list ["a", "b", "c"].

"hello".replace("l", "L") gives heLLo, replacing every l with L.

"hello".find("l") gives 2, the index of the first l in the string.

"123".isdigit() gives True while "abc123".isdigit() gives False, testing the content.

These methods return new strings; the original string is never modified.

26 What is string formatting? Explain with an example.

IMPORTANT

String formatting means inserting variable values neatly inside text.

The easiest way is an f-string: put f before the opening quote and write variables inside curly braces.

Example code:

```
name = "Aben"
marks = 95
print(f"{name} scored {marks} marks")
```

Python replaces {name} with Aben and {marks} with 95.

Output: Aben scored 95 marks.

You can even calculate inside the braces: f"{marks + 5}" gives 100.

Older styles are the format() method ("{} scored {}".format(name, marks)) and the % operator, but f-strings are the modern standard.

Formatting keeps output readable without messy + joining and str() calls.

27 Write the output of: s = 'Python'; print(s[1:4], s * 2).

IMPORTANT

First, s[1:4] slices the string from index 1 up to index 4 (not included).

Indexes 1, 2, 3 of "Python" are y, t, h, so this gives yth.

Remember: the slice stops before the end index, so the character at index 4 (o) is not included.

Next, s * 2 uses the repetition operator, which repeats the whole string twice.

"Python" repeated twice is PythonPython.

Note that * with a string repeats it, while * with numbers multiplies, so the context decides its meaning.

print() shows both values separated by a single space by default.

So the full output is: yth PythonPython.

28 **How do you access individual characters in a string? Explain with examples.**

IMPORTANT

Characters in a string are accessed by index in square brackets, starting from 0.

s[0] is the first character, s[1] the second, and so on.

Negative indexing counts from the end: s[-1] is the last character.

Example:

```
s = "Python"
print(s[0])    # P
print(s[-1])   # n
```

Going past the last index raises an IndexError.

Indexing with 0 to n-1 and negatives is the basic way to read single characters.

29 **Write a Python program to display the letters of a given string.**

IMPORTANT

A for loop can walk through a string one character at a time.

```
word = input("Enter a string: ")
for ch in word:
print(ch)
```

Each pass of the loop puts the next character into ch and prints it on its own line.

The same can be done with indexes: for i in range(len(word)): print(word[i]).

Looping directly over the string is the simplest letter-by-letter display.

30 **What will be the output of print(3 * 'Python' + '3')? Explain.**

IMPORTANT

* with a string repeats it: 3 * 'Python' makes 'PythonPythonPython'.

+ joins two strings: 'PythonPythonPython' + '3' gives 'PythonPythonPython3'.

So the output is PythonPythonPython3.

Note that '3' is a string here, not a number, so no arithmetic happens.

Rule to remember: string * number repeats, string + string joins.

31 **Explain break, continue and pass with examples. (Model Paper, MG2DSCCSC100)**

REPEATED

break ends the loop immediately, even if rounds remain.

continue skips the rest of the current round and jumps to the next round of the loop.

pass does nothing; it is a placeholder where Python needs a statement but you have nothing to write yet.

Example code:

```
for i in range(5):
if i == 3:
break
print(i)
```

This prints 0 1 2 and stops when i becomes 3.

If break were replaced with continue, it would print 0 1 2 4, skipping only 3.

pass is used in empty function or class bodies to avoid a syntax error.

Together, break, continue and pass give full control over how a loop behaves.

32 **Explain if-elif-else with an example.**

IMPORTANT

if, elif and else let the program choose which code to run based on conditions.

if checks the first condition; if it is True its block runs.

elif (short for else if) checks the next condition only when all previous ones were False.

else runs when every condition above it failed; it needs no condition of its own.

Example code:

```
marks = 75
if marks >= 90:
print("A grade")
elif marks >= 60:
print("B grade")
else:
print("C grade")
```

75 is not >= 90 but is >= 60, so the output is B grade.

You can have many elifs but only one else, and the first True block runs while the rest are skipped.

33 Distinguish between for and while loops with examples.

IMPORTANT

Both loops repeat code, but they are used in different situations.

for is used when the number of repeats is known; it walks over a sequence.

while is used when repeats depend on a condition; it runs until the condition becomes False.

Example code:

```
for i in range(3):
print(i)
```

This prints 0 1 2, because range(3) gives 0, 1, 2.

```
n = 3
while n > 0:
print(n)
n = n - 1
```

This prints 3 2 1, stopping when n reaches 0.

Use for for counting and scanning lists; use while for waiting on changing conditions like user input.

A while loop with a condition that never becomes False runs forever, so the loop variable must be updated inside.

34 What is nested control flow? Give an example.

IMPORTANT

Nested control flow means writing one if statement or loop inside another.

The inner block runs fully for every single round of the outer block.

Example code:

```
for i in range(1, 4):
for j in range(1, 4):
print(i, "x", j, "=", i * j)
```

The outer loop picks i = 1, then the inner loop runs j = 1, 2, 3 completely before i becomes 2.

This prints a small multiplication table: 1x1=1, 1x2=2, 1x3=3, 2x1=2 and so on.

Nesting if inside if works the same way: the inner if is checked only when the outer condition is True.

Deep nesting makes code hard to read, so two levels is usually enough.

Nested loops are essential for tables, patterns and grid-like data.

35 **Write a Python program to check whether a given year is a leap year.**

IMPORTANT

A year is a leap year if it is divisible by 4, except century years which must be divisible by 400.

```
year = int(input("Enter year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
print("Leap year")
else:
print("Not a leap year")
```

year % 4 == 0 checks divisibility by 4; year % 100 != 0 removes most century years.

year % 400 == 0 keeps century years like 2000 that really are leap years.

The combined condition handles all three rules in one if statement.

36 **Rewrite the following for loop as a while loop with the same output: for i in range(1, 6): print(i * '*').**

IMPORTANT

A for loop with range(1, 6) runs i through 1, 2, 3, 4, 5; a while loop needs a manual start, condition and increment.

```
i = 1
while i < 6:
print(i * "*")
i += 1
```

i = 1 sets the same start; while i < 6 keeps the same end; i += 1 moves to the next value.

The output is identical: *, **, ***, ****, *****.

Every for-range loop can be rewritten as while by managing the counter yourself.

37 **Write a Python program to print the first 10 even numbers using a for loop.**

IMPORTANT

Even numbers are 2, 4, 6 and so on; range(2, 21, 2) steps through them directly.

```
for i in range(2, 21, 2):
print(i)
```

range(2, 21, 2) starts at 2, stops before 21 and steps by 2, giving exactly 10 numbers.

Alternative: loop 1 to 20 and print i only when i % 2 == 0.

A stepped range is the cleanest way to generate even numbers.

38 **Write a Python program that accepts a number and prints whether it is positive, negative or zero.**

IMPORTANT

if-elif-else picks exactly one of the three cases.

```
n = int(input("Enter a number: "))
if n > 0:
print("Positive")
elif n < 0:
print("Negative")
else:
print("Zero")
```

The conditions are checked top to bottom and the first true branch runs.

Three-way tests like this are the textbook use of if-elif-else.

39 **List the decision making statements in Python.**

IMPORTANT

if: runs a block only when its condition is true.

if-else: picks one of two blocks based on the condition.

if-elif-else: picks one branch out of many ordered conditions.

nested if: an if statement placed inside another if for layered decisions.

Example:

```
marks = 75
if marks >= 90:
print("A grade")
elif marks >= 60:
print("B grade")
else:
print("C grade")
```

These four forms cover every branching need in a program.

40 Explain finite loop and infinite loop with examples.

IMPORTANT

A finite loop runs a fixed, known number of times and then stops by itself.

Example of a finite loop: for i in range(5): print(i) always ends after 5 passes.

An infinite loop has a condition that never becomes false, so it runs forever unless broken.

Example of an infinite loop:

```
while True:
print("runs forever")
break   # without this break it would never stop
```

Infinite loops are sometimes wanted (servers, menus) but usually signal a missing update or a wrong condition.

Finite loops end on their own; infinite loops need a break or a fixed condition to stop.

41 Write a Python program to display a star pyramid pattern.

IMPORTANT

Each row i prints (n-i) spaces then (2*i-1) stars, forming a centered pyramid.

```
n = 5
for i in range(1, n + 1):
print(" " * (n - i) + "*" * (2 * i - 1))
```

" " * (n - i) pushes the stars right so the pyramid is centered.

"*" * (2 * i - 1) grows the star count 1, 3, 5, 7, 9 down the rows.

Spaces for shape plus stars for width is the standard pattern trick.

42 Write a Python program to check whether a number is an Armstrong number.

IMPORTANT

An Armstrong number equals the sum of its digits each raised to the power of the digit count (153 = 1^3+5^3+3^3).

```
n = int(input("Enter number: "))
digits = str(n)
power = len(digits)
total = sum(int(d) ** power for d in digits)
if total == n:
print("Armstrong number")
else:
```

```
print("Not an Armstrong number")
```

len() of the digit string gives the power; each digit is raised to it and added.

Compare the computed sum with the original number to decide.

43 Write the syntax of for and while loops and print the factorial of n using both.

IMPORTANT

for syntax: for variable in sequence: with the loop body indented below it.

while syntax: while condition: with the body indented, plus code that moves toward ending.

Factorial with for:

```
n = int(input("Enter n: "))
fact = 1
for i in range(1, n + 1):
fact *= i
print("Factorial:", fact)
```

Factorial with while:

```
n = int(input("Enter n: "))
fact = 1
i = 1
while i <= n:
fact *= i
i += 1
print("Factorial:", fact)
```

Both versions multiply 1 to n; for manages the counter while while needs a manual i += 1.

MODULE 2

Functions and Data Structures

2

EXAM PATTERN (from real MGU question papers)

Exam: MG3SECBCA200 Python Programming - 50 marks, 1 hour 30 minutes (MGU Honours model paper, mgu.ac.in, Oct 2025).

Part A: 10 x 2 = 20 marks, answer ALL - short questions from every module.

Part B: answer any 3 of 5 x 5 = 15 marks - programs and detailed concept answers.

Part C: answer any 1 of 2 x 15 = 15 marks - one long essay or program.

Module 2 in the model paper: dict key-value, import forms, Numpy array (Part A); assert program, dictionary methods (Part B); list methods essay (Part C).

1 What are built-in functions? Explain any five with examples.

IMPORTANT

Built-in functions are ready-made functions that come with Python, so we can use them directly without importing anything or writing them ourselves.

They handle the most common jobs in every program, like showing output, checking data types, and working with numbers and collections.

print() shows output on the screen, for example print('Hello') displays Hello.

len() counts the items in a collection, for example len('hello') gives 5 because there are five letters.

type() tells the data type of a value, for example type(10) gives <class 'int'>.

range() creates a sequence of numbers, for example list(range(5)) gives [0, 1, 2, 3, 4], and it is mainly used with for loops.

max() and min() pick the largest and smallest value, for example max(3, 9, 4) gives 9 and min(3, 9, 4) gives 3.

sum() adds up all numbers in a collection, for example sum([1, 2, 3]) gives 6.

Example code:

```
print(len('hello'))
print(max(3, 9, 4))
print(sum([1, 2, 3]))
```

The output is 5, 9 and 6.

Because these functions are always available, they make programs shorter and save the programmer from rewriting common logic.

2 Explain range() with its three forms.

IMPORTANT

range() is a built-in function that creates a sequence of numbers, and it is most often used to control how many times a for loop runs.

It never includes the stop value; it always stops one number before it.

First form, range(stop), starts from 0 and goes up to stop minus 1, for example range(5) gives 0 1 2 3 4.

Second form, range(start, stop), starts from start and goes up to stop minus 1, for example range(2, 6) gives 2 3 4 5.

Third form, range(start, stop, step), jumps by step each time, for example range(1, 10, 2) gives 1 3 5 7 9.

The step can also be negative, which counts backwards, for example range(5, 0, -1) gives 5 4 3 2 1.

range() does not make a list directly; it makes a range object that produces numbers one by one, which saves memory.

Example code:

```
for i in range(3):
print(i)
```

This prints 0, 1 and 2 on separate lines.

So range() is the standard way to repeat an action a fixed number of times in Python.

3 Write a user-defined function to find the largest of two numbers.

IMPORTANT

Define with def, compare with if-else and return the bigger value.

```
def largest(a, b):
if a > b:
return a
else:
return b
print(largest(10, 25))   # 25
```

Calling largest(10, 25) runs the function body with a = 10 and b = 25.

The function returns the greater of its two arguments.

4 **Write a Python function to calculate and return the factorial of a given number. (MG2DSCCSC100, Mar 2024)**

REPEATED

Loop from 1 to n multiplying into a running total, then return it.

```
def factorial(n):
fact = 1
for i in range(1, n + 1):
fact *= i
return fact
print(factorial(5))   # 120
```

fact starts at 1 because multiplying by 0 would ruin the result; return sends the answer back.

Recursive version: return 1 if n <= 1 else n * factorial(n - 1).

The function builds n! step by step and returns the final product.

5 **What is recursion? Explain how a recursive function works with a program. (Nov 2025, Mar 2024)**

REPEATED

Recursion is when a function calls itself to solve a smaller version of the same problem.

Every recursive function needs a base case that stops the calling, otherwise it never ends.

Example: factorial(3) = 3 * factorial(2) = 3 * 2 * factorial(1) = 3 * 2 * 1 = 6.

```
def factorial(n):
if n <= 1:                  # base case
return 1
return n * factorial(n - 1) # recursive call
print(factorial(5))             # 120
```

Each call waits for the smaller call to finish, then multiplies and returns up the chain.

Base case plus a self-call on a smaller input is the whole recursion pattern.

6 How do you define and call a function? Explain with an example.

IMPORTANT

A function is a named block of code that does one job, and it lets us reuse the same logic many times without rewriting it.

A function is defined with the def keyword, followed by the function name, brackets with parameters inside, a colon, and then the indented body.

To use the function we call it by its name with actual values in brackets; this is called the function call.

The values passed at the call are called arguments, and the names written in the definition are called parameters.

Example code:

```
def add(a, b):
return a + b


print(add(4, 5))
```

Here add is defined with parameters a and b; calling add(4, 5) runs the body with a = 4 and b = 5.

The return statement sends the result 9 back to the caller, so print shows 9 as the output.

Without a call, the function body never runs; the definition only stores the code.

Functions keep the program organized, because each part of the work has its own name and place.

7 Explain local and global scope with an example.

IMPORTANT

The scope of a variable is the part of the program where that variable can be seen and used.

Local scope means the variable is created inside a function, so it works only inside that function and is destroyed when the function ends.

Global scope means the variable is created outside all functions, at the top level, so every function in the program can read it.

A function cannot change a global variable by a plain assignment; assignment inside a function creates a new local variable instead.

To change a global variable inside a function we must declare it with the global keyword.

Example code:

```
x = 10
```

```
def show():
y = 5
print(x + y)
show()
```

Here x is global, so the function can read it; y is local to show() and exists only while show() runs.

The output is 15.

Trying to use y outside the function would give a NameError, which proves that local variables stay hidden.

So scope decides the lifetime and visibility of every variable in the program.

8 What are default parameters? Give an example.

IMPORTANT

A default parameter is a parameter that already has a value fixed in the function definition itself.

If the caller passes a value for it, that value is used; if the caller skips it, the default value is used automatically.

Default parameters make functions flexible, because the common case works without any argument while special cases can still pass their own value.

In the definition, default parameters must come after all the normal parameters.

A common rule is to avoid using a changeable object like a list as a default value, because the same object is shared across all calls.

Example code:

```
def greet(name='Guest'):
print('Hello', name)


greet()
greet('Aben')
```

The first call skips the argument, so the default 'Guest' is used and it prints Hello Guest.

The second call passes 'Aben', which replaces the default, so it prints Hello Aben.

Thus default parameters save the caller from passing values that are the same in most calls.

9 What does return do? What happens without it?

IMPORTANT

The return statement ends a function immediately and hands a value back to the caller.

After return runs, no further lines of the function body are executed.

A function can return any value: a number, a string, a list, or even another function.

A function can have many return statements, but only the first one reached actually runs.

If a function has no return statement, or just a bare return, Python automatically returns the special value None.

That means a function without return still does its work, like printing, but it gives nothing back to the caller.

Example code:

```
def add(a, b):
return a + b


print(add(2, 3))
```

Here return sends 5 back, so the output is 5; if the return line were removed, the output would be None.

So return is the way a function communicates its result to the rest of the program.

10 Which keywords are used in functions? Explain with examples.

IMPORTANT

def creates a function: def add(a, b): defines add.

return sends a value back to the caller and ends the function.

lambda builds a small anonymous function: f = lambda x: x * 2.

global lets a function modify a variable defined outside it.

nonlocal lets a nested function modify its enclosing function's variable.

yield turns a function into a generator that produces values one by one.

Example:

```
def square(x):
return x * x
print(square(6))   # 36
```

def and return are the core pair; lambda, global, nonlocal and yield handle special cases.

11 **Write a Python program which defines three functions to calculate the area of a rectangle, square and circle based on the user choice.**

IMPORTANT

Three small functions plus a menu that calls the chosen one.

```
import math
def area_rect(l, b):
return l * b
def area_square(s):
return s * s
def area_circle(r):
return math.pi * r * r
print("1. Rectangle  2. Square  3. Circle")
ch = int(input("Choice: "))
if ch == 1:
l = float(input("Length: "))
b = float(input("Breadth: "))
print("Area:", area_rect(l, b))
elif ch == 2:
s = float(input("Side: "))
print("Area:", area_square(s))
elif ch == 3:
r = float(input("Radius: "))
print("Area:", area_circle(r))
else:
print("Invalid choice")
```

Each function holds one area formula; the if-elif menu routes to the right one.

Separate functions keep each formula clean and reusable.

12 What are command line arguments? Explain with sys.argv.

IMPORTANT

Command line arguments are values given to a program when it is started from the terminal, so the program can change its behaviour without editing the code.

They are typed after the program name, for example python prog.py 10 20 makes 10 and 20 the arguments.

Python collects these values in the list sys.argv from the sys module.

sys.argv[0] is always the name of the program file itself, and the real arguments start from sys.argv[1].

All values in sys.argv are strings, so we must convert them with int() or float() before doing maths.

Example code:

```
import sys
print('Program:', sys.argv[0])
print('First arg:', sys.argv[1])
```

Running python prog.py 10 20 prints Program: prog.py and First arg: 10.

If we need the number 10 for addition, we write int(sys.argv[1]).

Command line arguments are useful for scripts, tools and automation, where the same program must handle different inputs each time.

13 What is a lambda function? Give two examples.

IMPORTANT

A lambda function is a small unnamed function written in a single line, used when we need a short function only once.

Its syntax is lambda arguments: expression, where the expression is calculated and its result is automatically returned.

Unlike a normal function, a lambda body can contain only one expression; it cannot have statements or multiple lines.

The result of lambda is a function object, so we usually store it in a variable and then call it.

Example code:

```
square = lambda x: x * x
add = lambda a, b: a + b
print(square(6), add(2, 3))
```

The output is 36 5, because square(6) gives 36 and add(2, 3) gives 5.

Lambdas are most useful where a short function is passed to another function, for example key = lambda s: len(s) inside sorted() to sort strings by length.

They are also commonly used with map() and filter() to process lists without writing a full def function.

14 Explain the assert statement with an example.

IMPORTANT

The assert statement is a debugging tool that checks whether a condition is true while the program runs.

Its form is assert condition, message, where the message is optional.

If the condition is True, the program continues normally as if nothing happened.

If the condition is False, Python raises an AssertionError with the message and stops the program at that point.

assert is meant to catch the programmer's own mistakes during development and testing, not to handle normal user errors.

Example code:

```
age = int(input('Enter age: '))
assert age > 0, 'age must be positive'
print('Age accepted')
```

If the user types 25, the assertion passes and Age accepted is printed.

If the user types -5, the program stops with AssertionError: age must be positive.

So assert acts like a guard that fails loudly the moment the program reaches an impossible state.

15 What is the use of the sys module in Python?

IMPORTANT

The sys module gives access to interpreter-level information and functions.

sys.argv holds command line arguments: sys.argv[0] is the script name and the rest are user inputs.

sys.exit() stops the program immediately, which is useful on fatal errors.

sys.version shows the Python version; sys.path lists where Python searches for modules.

Example:

```
import sys
print(sys.argv)   # ['prog.py', '10'] when run as python3 prog.py 10
```

sys is the bridge between a running program and the interpreter around it.

16 What is a module? How do you import a user defined module?

IMPORTANT

A module is simply a Python file that contains functions, variables or classes which we want to reuse in other programs.

Modules let us write useful code once and then use it in many programs, instead of copying the same code everywhere.

To make a user defined module, we save our functions in a file, for example mymath.py containing def add(a, b): return a + b.

In the program that needs it, we write import mymath, and Python loads the file.

Then we call its functions with the module name as a prefix, for example mymath.add(4, 5) gives 9.

The module file must be in the same folder as the program, or in a folder Python searches, for the import to work.

Example code:

```
import mymath
print(mymath.add(4, 5))
```

The output is 9.

So modules are the basic building blocks for organizing large programs into clean, reusable parts.

17 Distinguish between import math and from math import sqrt.

IMPORTANT

Both statements bring the math module into the program, but they differ in what names they expose and how the functions are called.

import math loads the whole module, and every function must be called with the module name as prefix, for example math.sqrt(16) gives 4.0.

This form keeps the namespace clean, because it is always clear where sqrt comes from.

from math import sqrt loads only the sqrt function into the current namespace, so we call it directly as sqrt(16) without the math prefix.

This form is shorter to write, which is handy when a function is used many times.

The risk of the from form is a name clash: if our own program already has a name sqrt, the imported one silently replaces it.

Full import math is safer in big programs where names may clash; from-import is fine in small scripts for brevity.

In short, import math is the safer long form, while from math import sqrt is the shorter direct form.

18 **Define a package in Python.**

IMPORTANT

A package is a folder of related Python modules grouped under one name for easy import.

The folder contains an __init__.py file (which may be empty) marking it as a package.

Example layout: mypack/__init__.py, mypack/maths.py, mypack/text.py.

It is imported with import mypack.maths or from mypack import maths.

Packages keep big projects organized instead of dumping every module in one folder.

A package is a directory of modules plus __init__.py, imported by its dotted name.

19 **Differentiate between pop(), popitem() and clear() in dictionaries with suitable examples. (Model Paper, MG2DSCCSC100, Nov 2025)**

SURE MARK

pop(key) removes the entry with the given key and returns its value; it raises KeyError if the key is missing (unless a default is given).

popitem() removes and returns the last inserted key-value pair as a tuple.

clear() removes all entries, leaving an empty dictionary.

Example:

```
d = {"a": 1, "b": 2, "c": 3}
print(d.pop("b"))      # 2, d is now {"a": 1, "c": 3}
print(d.popitem())     # ('c', 3), last pair removed
d.clear()
print(d)               # {}
```

Use pop when you know the key and want its value; popitem for last-in cleanup; clear to reset the whole dict.

In short: pop removes by key, popitem removes the last pair, clear empties everything.

20 Distinguish between lists, tuples and dictionaries.

IMPORTANT

A list is an ordered collection that can be changed after creation, written in square brackets, for example [1, 2, 3].

A tuple is an ordered collection that cannot be changed after creation, written in round brackets, for example (1, 2, 3).

A dictionary is an unordered collection of key-value pairs, written in curly braces, for example {'name': 'Aben'}.

Lists support changes like l[0] = 9, while doing t[0] = 9 on a tuple raises a TypeError.

Dictionary items are fetched by key instead of position, for example d['name'] gives 'Aben'.

Lists are faster to modify; tuples are faster to access and can be used as dictionary keys because they are unchangeable.

Example code:

```
l = [1, 2, 3]
l[0] = 9
d = {'name': 'Aben'}
print(l, d['name'])
```

The output is [9, 2, 3] Aben.

As a rule, use lists for collections that change, tuples for fixed data, and dictionaries for labelled data.

21 Explain any five list methods with examples.

IMPORTANT

List methods are built-in functions of the list object that add, remove or rearrange its items.

append(x) adds x at the end of the list, for example [1, 2].append(3) makes it [1, 2, 3].

insert(i, x) puts x at position i and shifts later items right, for example [1, 3].insert(1, 2) makes it [1, 2, 3].

remove(x) deletes the first occurrence of x, for example [1, 2, 2].remove(2) leaves [1, 2].

pop() removes the last item and returns it, for example [1, 2, 3].pop() gives 3 and the list becomes [1, 2]; pop(i) removes the item at position i.

sort() arranges the items in ascending order, for example [3, 1, 2].sort() makes it [1, 2, 3].

reverse() flips the order of the items, for example [1, 2, 3].reverse() makes it [3, 2, 1].

All these methods except pop change the list in place and return None, so we must not assign their result back to the list.

Mastering these five methods covers almost all everyday list work in Python.

22 What are mutable and immutable objects? Give examples.

IMPORTANT

Mutable objects are objects that can be changed after they are created, without making a new object.

The main mutable types are list, dictionary and set, for example we can add or remove items from them freely.

Immutable objects are objects that can never be changed after creation; any operation that looks like a change actually makes a new object.

The main immutable types are tuple, string, int and float.

Example code:

```
l = [1, 2]
l.append(3)
t = (1, 2)
```

After this, l becomes [1, 2, 3] because lists are mutable, but there is no way to change t.

Trying t[0] = 9 raises a TypeError, which proves tuples are immutable.

This difference matters when objects are passed to functions: mutable objects can be changed by the function, while immutable ones stay safe.

23 Explain how lists behave when passed to a function.

IMPORTANT

When a list is passed to a function, Python passes the reference to the same list object, not a copy of it.

So the parameter inside the function and the original list outside both point to the very same object in memory.

Any change the function makes through the parameter, like lst[i] = x, directly changes the caller's original list.

This is different from immutable values like numbers, where the function only gets a copy of the value.

Example code:

```
def double(lst):
for i in range(len(lst)):
lst[i] = lst[i] * 2
```

```
nums = [1, 2, 3]
double(nums)
print(nums)
```

The output is [2, 4, 6], which proves the original list nums was changed by the function.

This behaviour is powerful but needs care: if we want the function to leave the original untouched, we must pass a copy like nums[:].

24 Show with an example how to add a key-value pair to a dictionary.

IMPORTANT

Assigning to a new key inserts the pair; assigning to an existing key updates it.

```
d = {"name": "Asha"}
d["age"] = 20          # new key: added
d["name"] = "Asha R"   # existing key: updated
print(d)               # {'name': 'Asha R', 'age': 20}
```

The update() method can add several pairs at once: d.update({"city": "Kochi"}).

d[new_key] = value is the standard insert-or-update step.

25 Discuss the difference between lists and tuples with examples.

IMPORTANT

Lists are mutable: items can be changed, added or removed after creation; they are written in square brackets.

Tuples are immutable: once created they cannot change; they are written in round brackets.

Example:

```
lst = [1, 2, 3]
lst[0] = 9        # allowed
tpl = (1, 2, 3)
# tpl[0] = 9      # TypeError: tuples cannot change
```

Lists suit collections that grow or change (marks, cart items); tuples suit fixed records (coordinates, dates).

Tuples are slightly faster and can be dictionary keys; lists cannot.

Mutability is the core difference, and it decides which one to use.

26 What is an empty dictionary? Give an example program.

IMPORTANT

An empty dictionary has no key-value pairs; it is the starting point you fill later.

It is created with {} or dict().

```
d = {}
print(d)            # {}
print(len(d))       # 0
d["id"] = 101       # now it holds one pair
print(d)            # {'id': 101}
```

Programs often start empty and add entries inside a loop, for example when counting word frequencies.

{} or dict() makes the empty container; assignments fill it.

27 How do you concatenate two lists? Give an example.

IMPORTANT

The + operator joins two lists into a new one; extend() appends all items of one list into another.

```
a = [1, 2]
b = [3, 4]
c = a + b
print(c)            # [1, 2, 3, 4]
a.extend(b)
print(a)            # [1, 2, 3, 4]
```

+ leaves both originals unchanged and makes a new list; extend() changes the first list in place.

Use + for a new combined list and extend() to grow an existing one.

28 What is indexing and negative indexing in tuples? Explain with examples.

IMPORTANT

Indexing reads tuple items by position starting at 0: t[0] is the first item.

Negative indexing counts from the end: t[-1] is the last item and t[-2] the one before it.

```
t = (10, 20, 30, 40)
print(t[1])     # 20
print(t[-1])    # 40
print(t[1:3])   # (20, 30), slicing works too
```

Out-of-range indexes raise IndexError, exactly like lists.

Zero-based forward indexes plus minus-one-based backward indexes cover every position.

29 Write a Python program to sort a group of strings into alphabetical order.

IMPORTANT

The sorted() function returns a new sorted list; the sort() method sorts a list in place.

```
words = ["mango", "apple", "cherry"]
words.sort()
print(words)   # ['apple', 'cherry', 'mango']
```

Sorting is alphabetical (technically by character code); sorted(words) would keep the original untouched.

For case-insensitive order use sort(key=str.lower).

sort() or sorted() arranges strings A to Z in one call.

30 Explain the math module with any five functions.

IMPORTANT

The math module is Python's built-in library of mathematical functions, loaded with import math before use.

It provides exact, ready-made maths so we do not have to write formulas ourselves.

math.sqrt(x) gives the square root, for example math.sqrt(16) gives 4.0.

math.pow(x, y) gives x raised to the power y, for example math.pow(2, 3) gives 8.0.

math.factorial(n) gives the factorial of a whole number, for example math.factorial(5) gives 120.

math.ceil(x) rounds up to the nearest whole number, for example math.ceil(4.2) gives 5, while math.floor(x) rounds down, for example math.floor(4.8) gives 4.

The module also gives constants like math.pi, which is 3.14159..., used in circle calculations.

Example code:

```
import math
print(math.sqrt(16), math.factorial(5), math.ceil(4.2))
```

The output is 4.0 120 5.

So the math module covers the everyday maths needs of most programs in one import.

31 What is Numpy? How is it better than lists for numbers?

IMPORTANT

Numpy is a popular Python library for fast numerical work, built around a special object called the array.

A Numpy array holds many numbers of the same type in one compact block, unlike a Python list which can mix types.

Python list loops run in slow Python code, while Numpy's operations run in fast compiled C code, so large calculations finish much quicker.

Arrays use less memory than lists because the numbers are stored tightly together without extra object overhead.

Numpy supports whole-array maths, so arr * 2 doubles every element at once with no loop to write.

It also gives ready functions like np.sum, np.mean, np.max and np.min for instant statistics.

Example code:

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr * 2)
```

The output is [2 4 6], showing the whole array doubled in one step.

Because of this speed and simplicity, Numpy is the base of data science libraries like Pandas and Scikit-learn.

32 Write code to create a Numpy array and find its sum and mean.

IMPORTANT

A Numpy array is created with np.array(), which turns a Python list into a fast Numpy array.

Once the array exists, np.sum() adds all its elements and np.mean() gives their average.

The convention is to import Numpy as np with import numpy as np before doing anything else.

Example code:

```
import numpy as np
arr = np.array([10, 20, 30, 40])
print('Sum:', np.sum(arr))
print('Mean:', np.mean(arr))
```

Here arr holds the four numbers 10, 20, 30 and 40.

np.sum(arr) adds them to 100, and np.mean(arr) divides by 4 to give 25.0.

The output is Sum: 100 and Mean: 25.0.

This pattern of creating an array and calling one-line statistics functions is the core of everyday Numpy use.

33 **Write Python code using Numpy to generate an array of even numbers between 10 and 20.**

IMPORTANT

numpy.arange(start, stop, step) builds evenly spaced values; a step of 2 gives even numbers.

```
import numpy as np
arr = np.arange(10, 21, 2)
print(arr)   # [10 12 14 16 18 20]
```

arange(10, 21, 2) starts at 10, stops before 21 and steps by 2.

arange with step 2 is the direct Numpy way to list even numbers.

MODULE 3

Files, Exceptions, Visualization

3

EXAM PATTERN (from real MGU question papers)

Exam: MG3SECBCA200 Python Programming - 50 marks, 1 hour 30 minutes (MGU Honours model paper, mgu.ac.in, Oct 2025).

Part A: 10 x 2 = 20 marks, answer ALL - short questions from every module.

Part B: answer any 3 of 5 x 5 = 15 marks - programs and detailed concept answers.

Part C: answer any 1 of 2 x 15 = 15 marks - one long essay or program.

Module 3 in the model paper: text vs binary file, try block (Part A); stacked vs grouped bar chart, 2D vs 3D plotting (Part B).

1 Explain the write(), writeline() and writelines() methods with examples. (Nov 2025)

REPEATED

write(s) writes a single string to the file; it does not add a newline by itself.

writelines(lines) writes a list of strings one after another, also without adding newlines.

Note: the file object has no writeline() method; the singular form in the question means write().

Example:

```
f = open("data.txt", "w")
f.write("Hello\n")
f.writelines(["Line1\n", "Line2\n"])
f.close()
```

Always close the file (or use with) so the data is actually saved to disk.

write() is for one string, writelines() for a list, and you add newlines yourself where needed.

2 Explain the different file opening modes in Python.

IMPORTANT

A file mode is the second argument to open() and tells Python what you want to do with the file.

"r" opens for reading only; the file must already exist, otherwise an error occurs.

"w" opens for writing; it creates the file if missing and erases old content if it exists.

"a" opens for appending; new data is added at the end and old content stays safe.

"r+", "w+" and "a+" allow both reading and writing; w+ erases the file while a+ appends to it.

Add "b" for binary mode, used for images and pickle files: "rb" reads bytes, "wb" writes bytes.

"x" creates a new file and fails if the file already exists, which avoids accidental overwrites.

Example code:

```
f = open("data.txt", "w")
f.write("Hello")
```

```
f.close()
```

Choosing the right mode protects your data, so always pick the mode that matches your task.

3 Write a program to read a file and count lines, words and characters.

IMPORTANT

The task is to open a text file, read its content, and count its lines, words and characters.

The with statement opens the file and closes it automatically when the block ends.

read() brings the whole file content into one string called text.

splitlines() cuts the string at every newline, so the length of that list is the line count.

split() cuts the string at every whitespace, so the length of that list is the word count.

len(text) counts every character, including spaces and newline characters.

Example code:

```
with open("data.txt", "r") as f:
text = f.read()
lines = text.splitlines()
words = text.split()
print("Lines:", len(lines))
print("Words:", len(words))
print("Characters:", len(text))
```

Each counter uses the natural Python way of measuring that unit of text.

4 Why is the with statement preferred for files?

IMPORTANT

When you open a file, Python must later close it to free the file and save buffered data.

If you forget close(), buffered writes may never reach the file, so data is lost silently.

If an error happens between open and close, the file stays open and locked.

with open("data.txt", "r") as f: opens the file and assigns it to f.

When the block ends, Python closes the file automatically, even if an error occurred inside.

No explicit close() call is needed, so the code is shorter and cleaner.

Example code:

```
with open("marks.txt") as f:
data = f.read()
```

Once the block finishes, the file is guaranteed to be closed.

Because with is both safer and shorter, it is the preferred way to work with files.

5 Define a text file and a binary file.

IMPORTANT

A text file stores human-readable characters (letters, digits, symbols) encoded as text, for example .txt, .py and .csv files.

A binary file stores raw bytes that are not meant to be read as text, for example images, audio and .pyc files.

Text files are opened in 'r' or 'w' modes; binary files need 'rb' or 'wb' modes.

Reading a binary file as text garbles the data, so the open mode must match the content.

Text files hold characters, binary files hold raw bytes, and the mode must match.

6 Write a Python script to open a file, write a few lines of text into it and then read its content.

IMPORTANT

Open in 'w' to write, close it, then reopen in 'r' to read back.

```
with open("notes.txt", "w") as f:
f.write("Line one\n")
f.write("Line two\n")
with open("notes.txt", "r") as f:
print(f.read())
```

The with statement closes the file automatically, so nothing is lost.

read() returns the whole content as one string; the print shows both lines.

Write mode creates the content, read mode brings it back.

7 Write a program to copy a text file to another file.

IMPORTANT

Read the source fully, then write that content into the destination file.

```
with open("source.txt", "r") as src:
data = src.read()
with open("copy.txt", "w") as dst:
dst.write(data)
print("Copied")
```

For big files copy line by line instead: for line in src: dst.write(line).

Read-all then write-all copies any text file exactly.

8 Explain how you read and write numbers from/to a file with an example.

IMPORTANT

Files store text, so numbers must be converted with str() when writing and int() or float() when reading.

```
with open("nums.txt", "w") as f:
f.write(str(42) + "\n")
with open("nums.txt", "r") as f:
n = int(f.read().strip())
print(n + 8)   # 50, proves n is a real number
```

Writing the raw number without str() raises a TypeError.

strip() removes the newline before int() converts the text back.

str() on the way out and int() or float() on the way in.

9 What is pickling? Explain dump() and load() with an example.

IMPORTANT

Pickling converts a Python object into a byte stream that can be stored in a file.

This lets you save complex objects like lists and dictionaries and use them again later.

Unpickling is the reverse: it reads the bytes back and rebuilds the original object.

Both steps need the pickle module and the file opened in binary mode ("wb" or "rb").

pickle.dump(obj, f) pickles obj and writes it to file f.

pickle.load(f) reads the next pickled object from file f and returns it.

Example code:

```
import pickle
data = {"name": "Aben", "marks": 95}
with open("data.pkl", "wb") as f:
pickle.dump(data, f)
with open("data.pkl", "rb") as f:
saved = pickle.load(f)
print(saved)
```

The dictionary comes back unchanged with all its keys and values.

Pickling is ideal for saving Python objects quickly, but pickle files are only safe with trusted data.

10 How do you read and write CSV files in Python?

IMPORTANT

CSV (Comma Separated Values) files store table data with commas between values and lines between rows.

The csv module handles reading and writing, including tricky cases like commas inside values.

To write: open the file with newline="" to avoid blank lines, then get a writer with csv.writer(f).

writerow([..]) writes one row from a list; writerows([..]) writes many rows at once.

To read: use csv.reader(f), which gives each row back as a list of strings.

Example code:

```
import csv
with open("marks.csv", "w", newline="") as f:
w = csv.writer(f)
w.writerow(["name", "marks"])
w.writerow(["Aben", 95])
with open("marks.csv", "r") as f:
for row in csv.reader(f):
print(row)
```

Every row arrives as a list like ["Aben", "95"], so convert numbers with int() when needed.

CSV is the standard way to exchange simple tables with Excel and other tools.

11 How do you read and write JSON files? Distinguish JSON from CSV.

IMPORTANT

JSON (JavaScript Object Notation) stores data as readable text in name-value form, like Python dictionaries.

json.dump(obj, f) writes a Python object such as a dict or list into a text file.

json.load(f) reads the file and rebuilds the Python object.

Example code:

```
import json
d = {"name": "Aben", "marks": 95}
with open("d.json", "w") as f:
json.dump(d, f)
with open("d.json", "r") as f:
print(json.load(f))
```

The file contains plain readable text like {"name": "Aben", "marks": 95}.

Difference 1: CSV holds flat row-column tables only, while JSON holds nested structures like lists inside dicts.

Difference 2: CSV is best for spreadsheet-style data and Excel; JSON is best for web data, APIs and settings files.

Difference 3: CSV rows are all plain strings, while JSON preserves types like numbers, booleans and lists.

12 Explain try-except-else-finally with an example.

IMPORTANT

An exception is a runtime error, such as dividing by zero or converting bad input, that crashes the program if unhandled.

try: holds the risky code that might fail.

except: runs only when an error happens; you can name the error type, like ZeroDivisionError, to catch.

else: runs only when no error happened in the try block, for code that depends on success.

finally: runs in every case, error or not, and is used for cleanup like closing files.

Example code:

```
try:
x = int(input("Enter a number: "))
print(10 / x)
except ZeroDivisionError:
print("Cannot divide by zero")
except ValueError:
print("Enter digits only")
else:
print("Division done")
finally:
print("Program over")
```

Zero input prints the friendly zero message instead of crashing.

Using all four blocks gives full control: handle errors, confirm success, and always clean up.

13 What is the raise statement? Give an example.

IMPORTANT

Sometimes your program must fail on purpose, like when input breaks your own rules.

The raise statement creates an exception at the exact point you choose.

Syntax: raise ErrorType("message"), for example raise ValueError("age cannot be negative").

After raise, Python stops the normal flow and jumps to the nearest matching except block.

Example code:

```
age = int(input("Enter age: "))
if age < 0:
raise ValueError("age cannot be negative")
print("Age:", age)
```

A negative age raises ValueError with the given message instead of continuing silently.

raise is used for validation: reject bad data early with a clear error message.

14 Explain the exception hierarchy in Python.

IMPORTANT

Exceptions in Python are organized as a class tree, where every error is an object of some class.

BaseException is the root of the tree; almost everything inherits from it.

Exception is the main branch, and all normal runtime errors hang below it.

Built-in errors include: ValueError (wrong value), TypeError (wrong type), ZeroDivisionError (divide by zero), FileNotFoundError (missing file), IndexError (bad list index), KeyError (missing dict key).

Your own errors are made by writing a class that extends Exception, like class MyError(Exception): pass.

An except block catches the named class and all its subclasses, so except Exception catches almost everything.

Because of this, always catch specific errors first and put the general Exception last, or it would swallow the specific ones.

Example: except ValueError catches only bad values, while a bare except Exception at the end catches the rest.

15 Write a program that handles division by zero gracefully.

IMPORTANT

Goal: read two numbers, divide them, and show a friendly message if the divisor is zero.

Wrap the division inside a try block so errors are caught instead of crashing the program.

except ZeroDivisionError catches division by zero and prints a friendly message.

except ValueError catches non-numeric input from int().

Example code:

```
try:
a = int(input("Enter numerator: "))
b = int(input("Enter denominator: "))
print("Result:", a / b)
except ZeroDivisionError:
print("Error: cannot divide by zero")
except ValueError:
print("Error: enter numbers only")
```

Entering 0 as the denominator prints "Error: cannot divide by zero" and the program ends calmly.

Handling errors this way makes programs user friendly instead of showing scary tracebacks.

16 How do you draw a line plot, bar chart and pie chart in matplotlib?

IMPORTANT

matplotlib.pyplot, usually imported as plt, is Python's standard library for drawing charts.

plt.plot(x_values, y_values) draws a line plot joining the given points.

plt.bar(categories, values) draws a bar chart with one bar per category.

plt.pie(values, labels=labels) draws a pie chart showing each part of a whole.

plt.show() displays the figure on screen; call it after setting up each chart.

Example code:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [10, 20, 15])
plt.show()
plt.bar(["A", "B", "C"], [5, 8, 3])
plt.show()
plt.pie([40, 30, 30], labels=["A", "B", "C"])
plt.show()
```

plt.xlabel(".."), plt.ylabel(".") and plt.title("..") label any chart and make it exam worthy.

Line plots show trends over time, bar charts compare quantities, and pie charts show shares of a whole.

17 Write code to plot sine and cosine curves on the same graph.

IMPORTANT

To draw smooth curves we need many x points, which np.linspace(0, 2 * np.pi, 100) gives.

np.sin(x) and np.cos(x) compute the sine and cosine of every point at once.

Calling plt.plot twice draws both curves on the same figure.

The label argument names each curve so the legend can identify them.

plt.legend() displays the label box on the graph.

Example code:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 2 * np.pi, 100)
plt.plot(x, np.sin(x), label="sin")
plt.plot(x, np.cos(x), label="cos")
plt.legend()
plt.show()
```

The result is a smooth sine wave and a smooth cosine wave drawn together.

The same pattern works for any two curves: compute points, plot each with a label, add the legend.

18 Write a short note on histograms.

IMPORTANT

A histogram is a bar-like chart that shows how numerical values are distributed across ranges.

plt.hist(data, bins=10) splits the data range into 10 equal bins and counts the values in each.

Each bar's height shows how many values fell into that range.

Tall bars reveal where most values cluster, and the shape reveals patterns like normal or skewed distributions.

Example code:

```
import matplotlib.pyplot as plt
marks = [45, 67, 78, 56, 90, 34, 72, 88, 61, 55]
plt.hist(marks, bins=5)
```

```
plt.xlabel("Marks")
plt.ylabel("Students")
plt.title("Distribution of Marks")
plt.show()
```

Where a bar chart compares fixed categories, a histogram discovers the shape of continuous data.

Histograms are widely used in statistics and data analysis to summarize large datasets.

19 Differentiate between a stacked bar chart and a grouped bar chart.

IMPORTANT

A grouped bar chart places bars for each category side by side, so values are compared directly.

A stacked bar chart piles the values of one category on top of another in a single bar, showing the total and its parts.

Example:

```
import matplotlib.pyplot as plt
x = ["A", "B"]
plt.bar(x, [5, 7])   # grouped-style: separate bars per value
plt.show()
```

Stacked bars suit part-to-whole views like sales split by region; grouped bars suit direct comparison.

Grouped means side by side for comparison, stacked means piled for totals and shares.

20 Compare 2D and 3D plotting in Python with examples.

IMPORTANT

2D plotting draws on flat x-y axes using matplotlib.pyplot functions like plot(), bar() and pie().

3D plotting adds a z axis through mpl_toolkits.mplot3d for surfaces and 3D scatter views.

Example:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [4, 5, 6])   # 2D line plot
plt.show()
```

2D covers most exam charts; 3D is used when the data truly has three dimensions to show.

2D is the everyday x-y chart, 3D adds depth for three-variable data.

21 **Define the turtle module in Python. What are the steps in plotting a diagram using turtle? Give an example.**

IMPORTANT

The turtle module gives a pen (the turtle) that draws lines as it moves, used for simple 2D graphics.

Steps: import turtle, create a pen, move it with forward(), left() and right(), then keep the window open.

Example:

```
import turtle
t = turtle.Turtle()
for _ in range(4):
t.forward(100)
t.left(90)      # draws a square
turtle.done()
```

penup() and pendown() control whether movement draws; color() and pensize() style the line.

Turtle turns movement commands into drawings, step by step.

22 **Write a Python program to check whether a number is prime.**

IMPORTANT

Logic: read a number and test whether any value from 2 up to its square root divides it.

If a divisor is found, the number is not prime and the loop stops early with break.

If the loop finishes without a divisor (the for-else), the number is prime.

A prime number is divisible only by 1 and itself.

Full program:

```
n = int(input("Enter a number: "))
if n < 2:
print(n, "is not prime")
else:
for i in range(2, int(n ** 0.5) + 1):
if n % i == 0:
print(n, "is not prime")
break
else:
print(n, "is prime")
```

Checking only up to the square root keeps it fast.

23 Write a Python program to find the power of a number using a function.

IMPORTANT

Logic: define a function power(base, exp) that multiplies base by itself exp times.

Start result at 1 and repeat the multiplication in a loop exp times.

The main code reads base and exponent, calls the function and prints the result.

Full program:

```
def power(base, exp):
result = 1
for i in range(exp):
result = result * base
return result
b = int(input("Enter base: "))
e = int(input("Enter exponent: "))
print("Result:", power(b, e))
```

Example: base 2, exponent 5 gives 32.

24 Write a Python function to check whether a number is even or odd.

IMPORTANT

Logic: a number is even when its remainder on division by 2 is zero.

The function check uses the % operator and returns the word even or odd.

The main code reads a number, calls check and prints the verdict.

Full program:

```
def check(num):
if num % 2 == 0:
return "even"
else:
return "odd"
n = int(input("Enter a number: "))
print(n, "is", check(n))
```

The % operator gives the remainder; remainder 0 means even.

25 **Write a Python program to find the sum of the alternating series 1 - 2 + 3 - 4 ... n.**

IMPORTANT

Logic: the series 1 - 2 + 3 - 4 ... adds odd terms and subtracts even terms.

Loop i from 1 to n, adding i when i is odd and subtracting i when i is even.

Example: for n = 5 the sum is 1 - 2 + 3 - 4 + 5 = 3.

The series 1 - 2 + 3 - 4 ... n adds odd terms and subtracts even terms.

Full program:

```
n = int(input("Enter n: "))
total = 0
for i in range(1, n + 1):
if i % 2 == 0:
total = total - i
else:
total = total + i
print("Sum of series:", total)
```

For n = 5 the answer is 1 - 2 + 3 - 4 + 5 = 3.

26 **Write a menu-driven Python program for string operations: length, uppercase, reverse and vowel count.**

IMPORTANT

Logic: read a string once, then ask the user to pick one of four operations from a menu.

Use if-elif to run length, uppercase, reverse (slicing s[::-1]) or vowel counting.

The vowel counter walks through the string and counts characters found in aeiouAEIOU.

Full program:

```
s = input("Enter a string: ")
print("1. Length  2. Uppercase  3. Reverse  4. Count vowels")
ch = int(input("Enter choice: "))
if ch == 1:
print("Length:", len(s))
elif ch == 2:
print(s.upper())
elif ch == 3:
print(s[::-1])
elif ch == 4:
```

```
vowels = "aeiouAEIOU"
count = 0
for c in s:
if c in vowels:
count = count + 1
print("Vowels:", count)
else:
print("Invalid choice")
```

s[::-1] is the shortcut for reversing a string.

27 **Write a Python program to find all occurrence indices of a character in a string.**

IMPORTANT

Logic: scan the string position by position and compare each character with the target.

Every matching index is printed immediately; a flag records whether anything matched.

If the flag is still False at the end, report that the character was not found.

Full program:

```
s = input("Enter a string: ")
ch = input("Enter character to find: ")
found = False
for i in range(len(s)):
if s[i] == ch:
print("Found at index:", i)
found = True
if not found:
print("Character not found")
```

The loop checks every index and prints each match.

28 **Write a menu-driven Python program using Numpy to find sum, mean, max and min of an array.**

IMPORTANT

Logic: store the data in a Numpy array once, then offer sum, mean, max and min on a menu.

The choice number selects which Numpy function (np.sum, np.mean, np.max, np.min) to run.

Numpy computes each statistic over the whole array in a single call.

Full program:

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print("1. Sum  2. Mean  3. Max  4. Min")
ch = int(input("Enter choice: "))
if ch == 1:
print("Sum:", np.sum(arr))
elif ch == 2:
print("Mean:", np.mean(arr))
elif ch == 3:
print("Max:", np.max(arr))
elif ch == 4:
print("Min:", np.min(arr))
else:
print("Invalid choice")
```

Numpy functions work on the whole array at once.

29 **Write a Python program to count the frequency of each letter in a string using a dictionary.**

IMPORTANT

Logic: build a dictionary mapping each letter to its count.

Walk through the lowercased string, skipping spaces; add a new key for unseen letters or add 1 for seen ones.

Finally print every letter with its count from the dictionary.

Full program:

```
s = input("Enter a string: ").lower()
freq = {}
for c in s:
```

```
if c != " ":
if c in freq:
freq[c] = freq[c] + 1
else:
freq[c] = 1
for letter, count in freq.items():
print(letter, ":", count)
```

lower() makes counting case insensitive; spaces are skipped.

30 **Write a Python program to demonstrate tuple operations: length, max, min, sum, index, count and sorting.**

IMPORTANT

Logic: create a fixed tuple and demonstrate the operations allowed on it.

Built-ins len, max, min and sum work directly; index and count search the tuple.

sorted() returns a list, so wrap it in tuple() to get a sorted tuple back.

Full program:

```
t = (5, 2, 9, 1, 7)
print("Tuple:", t)
print("Length:", len(t))
print("Max:", max(t), "Min:", min(t))
print("Sum:", sum(t))
print("Index of 9:", t.index(9))
print("Count of 2:", t.count(2))
print("Sorted:", tuple(sorted(t)))
```

sorted() returns a list, so tuple() converts it back.

31 **Write a Python program to copy alternate lines from one file to another.**

IMPORTANT

Logic: read all lines of the source file into a list with readlines().

Indices 0, 2, 4, ... are the alternate lines, picked with range(0, len(lines), 2).

Write only those lines into the destination file.

Full program:

```
with open("source.txt", "r") as f1:
lines = f1.readlines()
```

```
with open("dest.txt", "w") as f2:
for i in range(0, len(lines), 2):
f2.write(lines[i])
print("Alternate lines copied to dest.txt")
```

range(0, len(lines), 2) picks indices 0, 2, 4, the alternate lines.

32 Write a Python program to handle ZeroDivisionError using exception handling.

IMPORTANT

Logic: read numerator and denominator and divide them inside a try block.

If the denominator is zero, ZeroDivisionError is caught and a friendly message prints.

ValueError handles the case where the user types non-numeric input.

Full program:

```
try:
a = int(input("Enter numerator: "))
b = int(input("Enter denominator: "))
print("Result:", a / b)
except ZeroDivisionError:
print("Error: cannot divide by zero")
except ValueError:
print("Error: enter numbers only")
```

The specific ZeroDivisionError is caught before the general ValueError case.

33 Write a Python program that defines a custom exception for duplicate entries and handles it.

IMPORTANT

Logic: define DuplicateError as a class that extends Exception.

While collecting items, check each new entry against the list collected so far.

If an entry repeats, raise DuplicateError, which the except block catches and reports.

A custom exception is made by writing a class that extends Exception.

Here DuplicateError is raised when the same item is entered twice.

Full program:

```
class DuplicateError(Exception):
pass
items = []
```

```
n = int(input("How many items: "))
try:
for i in range(n):
x = input("Enter item: ")
if x in items:
raise DuplicateError("Duplicate entry: " + x)
items.append(x)
print("List:", items)
except DuplicateError as e:
print("Error:", e)
```

raise triggers the exception; except catches it by name.

34 Write a Python program to plot sine and cosine curves using matplotlib.

IMPORTANT

Logic: create 100 smooth x points from 0 to 2 pi with np.linspace.

Plot np.sin(x) and np.cos(x) on the same figure with labels, then add legend, grid and titles.

plt.show() displays both curves together.

Full program:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 2 * np.pi, 100)
plt.plot(x, np.sin(x), label="sin(x)")
plt.plot(x, np.cos(x), label="cos(x)")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Sine and Cosine Curves")
plt.legend()
plt.grid(True)
plt.show()
```

linspace gives 100 smooth points between 0 and 2 pi.

35 **Write a Python program to draw a line plot of monthly profit.**

IMPORTANT

Logic: store month names and matching profit values in two parallel lists.

plt.plot joins the monthly points with a line; marker="o" marks each data point with a dot.

Labels and a title explain the axes before plt.show() displays the chart.

Full program:

```
import matplotlib.pyplot as plt
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
profit = [12000, 15000, 13000, 18000, 20000, 17000]
plt.plot(months, profit, marker="o")
plt.xlabel("Month")
plt.ylabel("Profit (Rs)")
plt.title("Monthly Profit")
plt.grid(True)
plt.show()
```

marker="o" puts a dot on each data point.